don't fully support CORS but do support `<iframe>`-to-`<iframe>` communication via `postMessage`. As you saw in chapter 2, Internet Explorer 8 and Internet Explorer 9 have limited support for CORS, so if those browsers are important to you, the `postMessage` technique may be a good alternative.

An alternative option is the easyXDM library, which can be found at http://easyxdm.net/. The easyXDM library provides a consistent interface for making cross-origin requests across all browsers. The library uses the `postMessage` technique for browsers that support `postMessage`, and falls back on other techniques (such as Flash) for browsers that don't support `postMessage`. The easyXDM library handles the complex details of making a cross-origin request so you don't have to.

D.4 Server-side request

Unlike JavaScript code running in a browser, code running on a server isn't constrained by the same-origin policy. If none of the other techniques work for your cross-origin needs, you can always write server-side code to make a cross-origin request, as shown in figure D.8.

Server code can make a request to any web page, regardless of whether that web page has CORS headers or not. Figure D.9 shows what a request flow looks like when using a server to make cross-origin requests:

1. The client initiates the request by making an `XMLHttpRequest` to the client's own web server. This is different from CORS, where the `XMLHttpRequest` was made directly to the remote server ❶.
2. The client's server makes a request to the API server and receives the response ❷.
3. The client's server forwards the response back down to the client's JavaScript code. Because the request from the client's JavaScript code to the client's server all takes place on the same origin, there is no need for CORS headers on the response ❸.

Making server-side requests can enable almost any type of cross-origin request. Because this is such a powerful technique, remote servers may not look too kindly on this technique, and may even take steps to block clients from making these types of requests. If you make cross-origin requests from the server, be sure to get the remote server's permission to do so. The one thing server-side requests can't do is include cookies from the remote origin. If the remote origin saved any cookies in the user's browser, there is no way for the client JavaScript code to read those cookies and include them on the request.

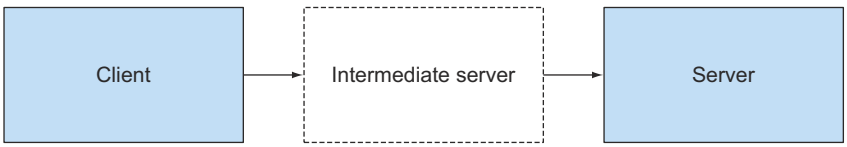


Figure D.8 Server-side code isn't constrained by the browser's same-origin policy.